Note that I rearranged the preceding YAML to increase its readability and omitted additional fields that are not relevant to our discussion.

I want to repeat this: Secrets are not secret! I can decode the value stored in the `test-credentials` Secret with a simple command:

```
$ echo 'cGFzc3dvcmQ=' | base64 --decode
password
```

Like ConfigMaps, Secrets can be passed to a container as environment variables or through a volume mount. In the second case, you can mount them as property files or config trees. For example, the `test-credentials` Secret would be mounted as a config tree because it's composed of key/value pairs rather than a file.

Since Secrets are not encrypted, we can't include them in a version control system. It's up to the platform engineers to ensure that Secrets are adequately protected. For example, Kubernetes could be configured to store Secrets in its internal *etcd* storage encrypted. That would help ensure security at rest, but it doesn't solve the problem of managing them in a version control system.

Bitnami introduced a project called *Sealed Secrets* (https://github.com/bitnami-labs/sealed-secrets), aimed at encrypting Secrets and putting them under version control. First you would generate an encrypted `SealedSecret` object, starting from literal values, similar to what we did for the plain Secret. Then you would include that in your repository and safely put it under version control. When the `SealedSecret` manifest is applied to a Kubernetes cluster, the Sealed Secrets controller decrypts its content and generates a standard Secret object that can be used within a Pod.